SheetShrink: Efficient Spreadsheet Cleanup

Alexander Aue-Johr
Institute of Rural Studies
Thünen-Institute
Braunschweig, Germany
Harz University of Applied Sciences
Doctoral Candidate, Germany
ORCID 0009-0001-8683-3630

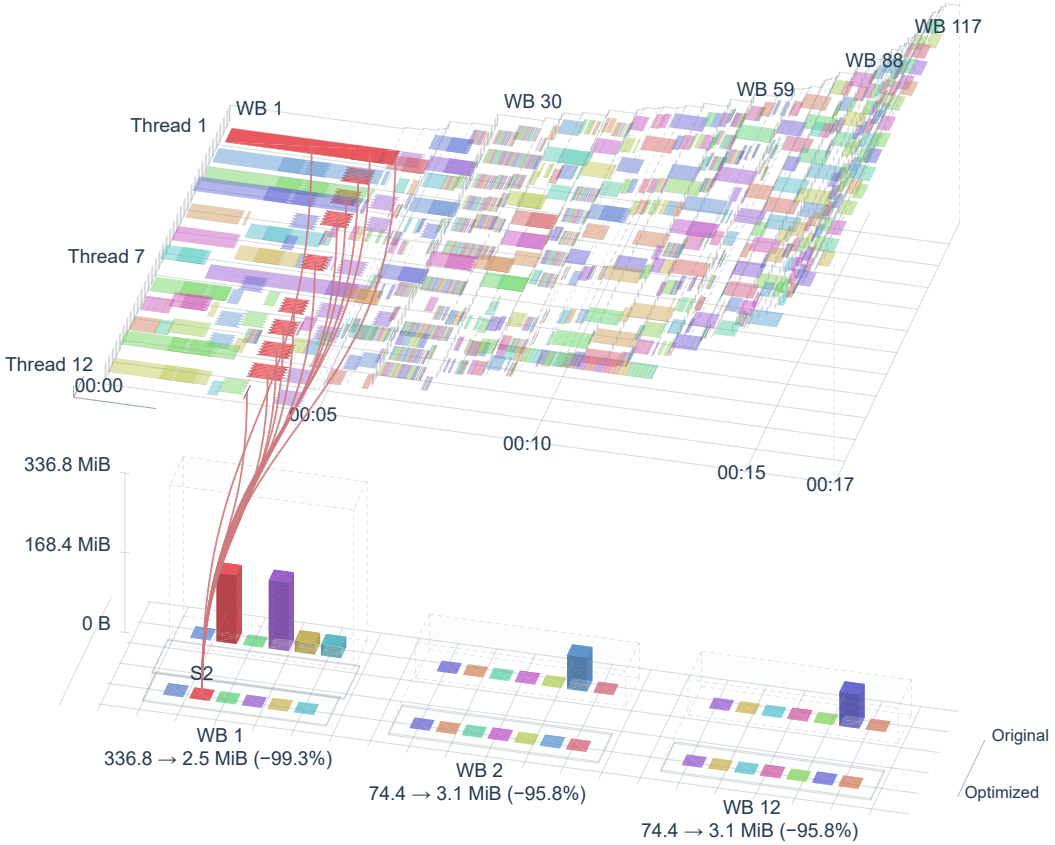


Fig. 1. Recorded development run on 144 Enron workbooks selected for large absolute XLSX reductions, using 12 workers. Work intervals follow workbook admission order (WB denotes workbook, numbered by admission). Thread labels identify worker lanes. The lower view compares the three largest uncompressed packages on one linear byte scale. Curves connect worksheet S2 in WB 1 to its work items. Observed wall time was 17.077 s.

***Abstract*—Empty and redundant cell records can make XLSX files expensive to store and load. SheetShrink removes redundant cell and formatting records directly from XLSX packages before they are opened in Excel. Its removal rules account for inherited formats, automatic row heights, and references from formulas and charts. A Rust implementation scans worksheet XML as bytes and processes worksheets in parallel. We evaluate cleanup on 1,182 Destatis, 8,623 Data.gov, and 30,301 prepared Enron workbooks. Aggregate compressed file sizes decrease by 15.48%, 5.29%, and 18.92%, respectively. A further 494 Data.gov files use unsupported XML namespaces and are skipped. Opening and saving experiments measure how cleanup changes processing time and memory use in spreadsheet libraries and Excel. Native Excel comparisons and controlled regression cases examine changes in appearance, including empty cells whose fonts or borders affect the rendered worksheet.**

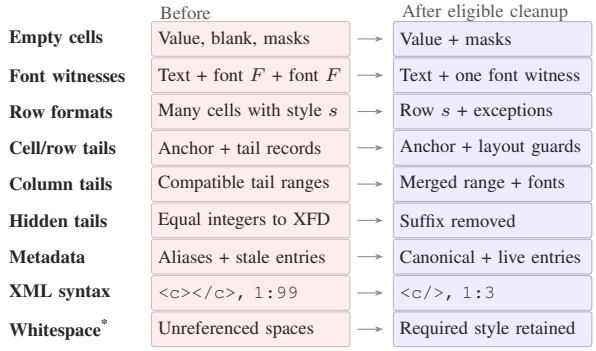


Fig. 2. Schematic cleanup families. Redundant cell formatting includes inherited fills and borders. Every pair has layout, protection and dependency guards. Row promotion requires compatible fixed-height context. Hidden-value removal admits guarded constant tails of at least 4,096 cells ending at XFD. *Whitespace removal is optional. Required witnesses and cell coordinates are retained.

I. Introduction

An XLSX workbook can contain a small table alongside many empty cell records and redundant row and column records. Decompressing and parsing these records adds work. Removing every empty-looking record is unsafe. An empty cell can preserve a border, override an inherited fill, or influence automatic row height through its font. A whitespace value can prevent text from extending into its neighbour. Cleanup therefore requires a model of the effects that stored structures have on the document.

A motivating case is the Destatis migration workbook `wanderungen-2010120217005.xlsx`. In worksheet `Tabelle 3.4`, 21 rows each contain numeric zeros from column L through Excel's final column XFD. Those tails alone contain 343,833 stored values. They are hidden content, so removing them requires a stated admission policy and dependency checks in addition to empty-format cleanup. Section III-E describes the case and the values that the conservative policy deliberately retains.

For an XLSX input, SheetShrink makes these decisions from the stored package before loading the workbook in Excel. Its cleanup stage has no native rendering or calculated row-height observations. An Excel-hosted method can inspect the loaded layout and evaluate candidate edits against native observations. Our method instead retains compact source structures whose required effects follow from package analysis and the preservation rules. Native Excel is used later to evaluate the output.

Specialized spreadsheet parsing has demonstrated the importance of decompression, parsing, memory use and parallel execution for data loading [1]. Our problem adds a transformation obligation. The output remains an editable XLSX workbook and must retain required data, dependencies and visible presentation under a stated contract. File size alone is an inadequate measure of success.

Figure 2 summarizes guarded reductions across cells, rows, columns and workbook metadata. SheetShrink reuses existing style indices and compact witnesses to preserve required effects. Native saving can additionally compact style and shared-string tables.

The performance objective includes the complete user operation. For a sufficiently bloated workbook, we seek to make cleanup followed by opening the cleaned file faster than directly opening its original. A single-thread comparison separates improvements in work performed from gains due to extra processing resources. Parallel corpus experiments then examine throughput and memory.

II. Preservation and Comparison Contract

A. Observable spreadsheet behavior

A spreadsheet package contains more than the populated table that a user sees. Individual cell records can carry formatting, protection, references, or values that affect other cells. Conversely, the presence of a stored record does not establish that it contributes to the intended presentation. SheetShrink targets production cleanup of such packages while retaining the structures needed by its preservation contract. Aggressive historical configurations are separate test artifacts and are excluded from the production subject of this paper.

The relevant visual area includes cells with meaningful values or formulas, empty cells with effective visible fills or borders, and graphical objects. Formatting inherited from rows or columns can contribute to this area. Conditional formatting and text extending into adjacent cells can also affect what is visible. The union of the original and transformed relevant areas is required when checking removed content. Layout changes in otherwise empty trailing rows or columns are permitted by the contract. Changes that move or alter relevant content remain subject to validation.

B. Information available before native loading

The cleanup decision uses stored values, formulas, styles, references and layout attributes. Native effective row heights and candidate before/after images are available only in the separate validation workflow. Stored custom heights can be retained directly. For automatically sized rows, a font-bearing blank can still require a representative even when its payload is empty.

An Excel-hosted alternative could measure a row, remove candidates and write the measured height as an explicit custom height. SpreadsheetML provides a *customHeight* attribute for that policy [2].

C. Removal decisions and compact preservation

SheetShrink retains a representative record when it preserves a required effect. Examples include row-font sentinels and reuse of an existing cell style as a row format. The checks account for inherited formatting, gaps, merges, protection and available styles. Unresolved cases retain the required original records.

The production policy removes hidden constant tails only after dependency and layout checks [3]. Ordinary nonempty hidden tables remain intact, including unreferenced values. A

missing reference alone is insufficient grounds for deleting hidden data.

D. Validation and attribution boundaries

DeltaSieve compares selected regions of independently rebuilt original and SheetShrink workbooks in the same installed Excel environment [4]. Ordinary-save and rebuild controls retain information for attributing differences introduced by Excel.

For XLS sources, conversion precedes the XLSX cleanup subject and has its own failure accounting. A file requiring repair or inaccessible protected content receives an explicit unsuccessful outcome.

Matching images establish equality within the captured regions. We evaluated region selection using independent native reference captures and deliberately modified test workbooks [5], [6]. We retain pixel differences and investigate their causes through repeated rendering, volatile dependencies and capture geometry.

III. Method

A. Representation and analysis stages

The input is an XLSX ZIP package. The output retains its workbook structure and cell coordinates while removing records admitted by the preservation policy. Package metadata supplies style components, shared strings, relationships, merged regions and dependency sources. These contexts are prepared before cell-local decisions, so an index or an empty payload is never sufficient by itself to establish redundancy.

Worksheet analysis consumes decompressed byte buffers without constructing an XML object for every cell. A state machine separates worksheet headers, column records, sheet data, merge information and the remaining suffix. In cell and row regions, bounded tag and attribute readers track hierarchy and quoting. Thus text resembling a coordinate inside a quoted metadata value cannot become a cell address. Small overlap state recognizes markers spanning input-buffer boundaries. Parsed metadata is retained only where later decisions require it. Names, references and other interpreted values can still require decoding. The implementation does not assume that arbitrary worksheet text is ASCII.

This organization reduces per-record allocation, but does not imply constant memory in worksheet size. The current implementation retains the sheet-data bytes, contexts and candidate metrics needed by subsequent passes. Decompression, rewriting, compression and retained analysis structures all contribute to peak memory. The supported input domain is normal unprefixed Transitional SpreadsheetML. Unsupported namespaces, encodings and ambiguous core structures follow explicit preservation or exclusion paths [3]. Eligibility checks inspect bounded package-root headers and declarations encountered during streaming to identify unsupported representations before native opening.

Figure 3 makes the worksheet preprocessor's control states explicit. The dispatcher separates column, cell and merge contexts. Read boundaries preserve partial state, and complete

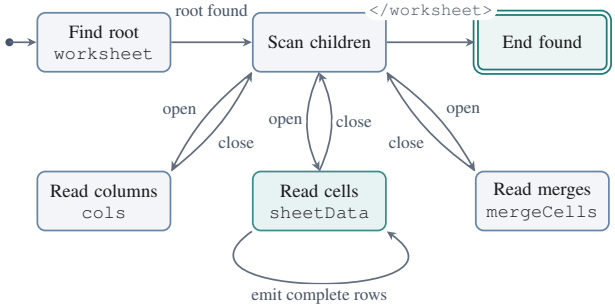


Fig. 3. Worksheet-stream states. Child readers return to the dispatcher; read boundaries preserve state and partial input. Reconstruction requires a completed worksheet; incomplete input, I/O failures and unsupported XML return an error.

row prefixes can progress without consuming an unfinished row. Reconstruction follows successful arrival at the worksheet end.

B. Parser correctness and regression controls

The byte scanner must recognize XML structure and track parser state. It must check element boundaries, quoted attributes, empty elements and incomplete input. Established SAX and DOM parsers provide extensively tested syntax handling. Memory-safe code can still misclassify content and produce an incorrect workbook.

Our development history supplies a concrete example. A Data.gov workbook used `<v xml:space="preserve">` for visible string values. A scanner recognizing only the literal `<v>` classified these cells as empty and removed a heading and other strings. The correction recognizes valid element-name delimiters and retains attributed value and inline-string elements. Negative controls reject longer lookalike names such as `<value>` [7]. A separate defined-name regression ensures that a self-closing element cannot consume the following paired element during replacement [8].

The worksheet stream must reach its closing state before output publication. An incomplete parse raises an error, removes partial output and preserves any existing destination. Integration fixtures exercise this failure after another worksheet has already been processed [9]. Streaming diagnostics additionally test comments, CDATA, quotes, nesting limits and source offsets across every input chunk size of their small fixtures.

At the time of writing, 31 correctness tests in the dedicated XML, namespace, shared-string, range, defined-name and streaming-reader modules exercise parsing and recognition behavior. A further 155 correctness tests in the cell, row and tail suites cover preservation witnesses, inheritance masks and content boundaries. Additional tests cover dependencies, styles, output construction and execution behavior. Counts refer to test functions, with parameterized fixture iterations counted once. The artifact records the source revision, test paths and counting criteria.

Source

```
<row r="2">
 <c r="A2">...</c>
 <c r="B2" s="1"/>
- <c r="C2" s="1"/>
</row>
```

Guarded result

```
<row r="2">
 <c r="A2">...</c>
 <c r="B2" s="1"/>
</row>
```

Original

Font witness control			
10 pt text			
Next visible row			

One witness

Font witness control			
10 pt text			
Next visible row			

No witness

Font witness control			
10 pt text			
Next visible row			

Fig. 4. Font-witness control. A2 uses Arial 10; empty B2 and C2 use Arial 18. Retaining either blank preserves the rebuilt rendering. Removing both reduces row 2 from 23.25 to 12.75 points and shifts the following row. XML is abbreviated; images are native Excel print exports.

C. Effects and compact preservation witnesses

A numeric value, formula or inline string is content. In particular, zero and a shared-formula follower with no independent formula text remain content. An empty cell is classified by its effective formatting in the cell, row and column context. An explicit no-fill or borderless style can suppress a visible inherited effect. It remains necessary whenever deletion would expose that effect. Conversely, an explicit no-fill with nothing to clear can be redundant. Style components and application flags are therefore interpreted together.

An empty cell's font can set automatic row height and shift later content. SheetShrink keeps a required representative for repeated font records. Equivalent fonts can share an identity despite different style indices. Column-font masks and merged spans also constrain removal. A record elsewhere in the row may not preserve a required local mask. Figure 4 isolates the mechanism in a constructed native Excel control. One witness preserves the observed height. Removing both moves the next visible row.

A fixed height eliminates this automatic-height effect, but an empty font-bearing cell can still affect the text baseline in a row containing text. The cleaner therefore distinguishes an empty fixed-height row from a fixed-height text row. Protection metadata does not erase a required font contribution. Protection formats inside the retained data extent also preserve interaction semantics, while separately guarded exterior tails can be removed.

Other witnesses preserve relationships between coordinates. Center-across-selection requires the contiguous blank span following the text cell. A borderless row can mask column borders, and an empty height-bearing row can determine their vertical extent. Column widths remain necessary when they support visible row borders. A chart can inherit a number format from an empty first series cell. These structures extend the preservation problem beyond the last nonempty value. Figure 5 shows an Enron counterexample in which Check Performance's smaller output exposes a different inherited fill. The observation concerns the pictured rebuilt region.

D. Guarded promotion and tail pruning

SheetShrink can replace repeated cell styles with an existing cell style applied to the row. The row must be visible, unprotected and have no row style. A stored height and enabled `customHeight` flag are required. The `ht` attribute alone permits automatic sizing. Content, merges, gaps and inherited column formats must remain compatible with the row format. Figure 6 illustrates the full-width case [10].

The dominant empty-cell style occurs at least twice. Selection counts unsuitable styles too, then applies eligibility checks. Otherwise an eligible minority could incorrectly replace an unsuitable majority. Ties choose the smaller existing style index. Dense promotion requires at least 1,024 occurrences. A bounded-gap route admits at most 64 missing columns and restores required original formats only under its inheritance checks. Sparse promotion requires equivalent inheritance at every absent position and excludes merged sparse rows. Uniform-fill promotion additionally checks content-cell fills. Unknown metadata or style identities prevent promotion and retain the ordinary cleanup path.

Pruning removes formatting-only records beyond the last required content or layout anchor. Font-only blanks beyond this boundary have no later content to shift. Fills, borders, conditional formatting and drawings can extend the boundary. A white row fill is redundant only if compatible column fills cover every column through XFD. Compatible column ranges can be merged while preserving local overrides and geometry needed for borders.

E. Hidden content and the Wanderungen case

SheetShrink uses a conservative heuristic to remove certain hidden numeric tails. A candidate must contain at least 4,096 identical integers and extend to XFD. This restriction targets large repeated tails such as those in Wanderungen. Candidates must also pass cell-level layout and dependency checks. Gaps, formulas, text, differing values and unsupported numeric forms reject the band. Source cells contribute before style cleanup. The common zero path compares value bytes directly, while one admitted row span replaces per-cell entries in the later reference map. Temporary row-local candidates still exist.

The motivating migration workbook contains 21 such zero tails in `Tabelle 3.4`: rows 11–30 and 32, from L through XFD. Each contains 16,373 zeros, totaling 343,833. Source

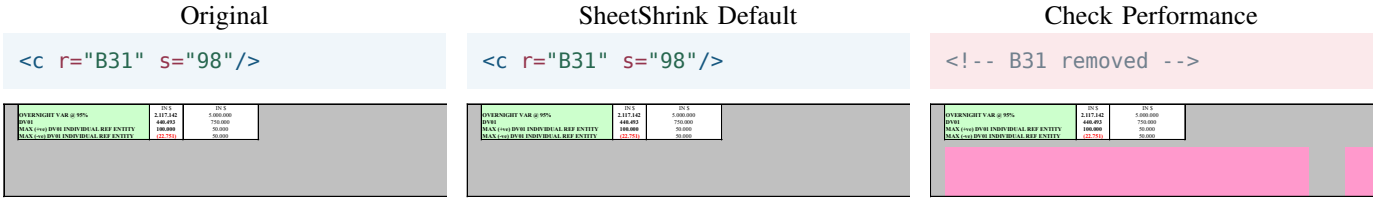


Fig. 5. An empty cell masks inherited formatting. In the Enron example, gray B31 overrides a pink column fill. SheetShrink preserves the mask; Check Performance removes it and exposes pink. Native print exports match for Original and SheetShrink after full rebuild within the displayed region. XML is abbreviated.



Fig. 6. Guarded row-style promotion replaces 16,383 repeated empty-cell records with an existing row format while retaining the value cell. Red marks removals and green additions. Intermediate cells are omitted. Promotion requires fixed-height, protection and inheritance checks described in the method. Style indices are local.

ZIP inspection confirms these counts and retains the source hash. The historical cleanup investigation records 376,573 removed zeros across two worksheets, alongside 243 preserved nonzero values and a retained short 237-zero tail [11]. These are content removals under an explicit structural policy. A deliberate constant full-width matrix can meet the same heuristic, so repetition alone does not establish user intent.

F. Workbook dependencies and deferred rewriting

SheetShrink postpones reference-sensitive deletions until it has checked supported formulas, names, charts, objects and pivot dependencies across the workbook. This includes references from later worksheets. Workbook relationships and sheet scope identify reference targets. Shared-formula followers shift relative axes and preserve absolute axes. The implementation protects enclosing referenced ranges without enumerating each shared-formula follower's references. Dynamic or unresolved references and unsupported expressions block deletion [12].

A known reference intersecting an admitted hidden span protects the entire span. Only admitted candidates reach rewriting. Removing a payload need not remove its format if that format still masks an inherited effect. After deferred changes, affected rows lose their stale tail anchors and pass through cleanup again. Otherwise records retained solely because of the removed anchor would survive incorrectly. Whitespace removal is disabled in Default because whitespace can control text overflow and row layout.

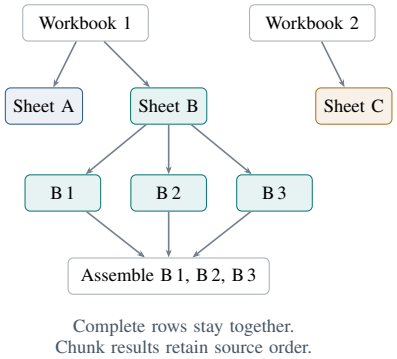


Fig. 7. Task hierarchy. Workbook admission bounds active inputs; sheets and row-aligned chunks share the Rayon pool. B 1–B 3 belong to one worksheet and are assembled in source order. Colors match Figure 8.

Package metadata has separate predicates. Calculation-chain cleanup resolves removed cells by sheet identity and preserves unresolved mappings. The cleaner does not evaluate formulas or manufacture replacement caches. Built-in alias cleanup currently matches name and local scope without proving equality of the two formula bodies.

G. Parallel work and output ownership

Figures 7 and 8 show the task hierarchy and a schematic execution across threads. Worksheet identity remains shared across its chunks, while ordered assembly separates execution order from output order. Workbook, worksheet and row-chunk work share the cleanup execution budget. A permit limits active workbooks and remains held through output finalization. Workers can process other workbooks while one long worksheet remains active. Large worksheets expose row-aligned chunks to Rayon, allowing idle workers to help without splitting a row's font, promotion or tail decisions. The chunk planner estimates byte-sized work, then adjusts boundaries to complete rows. Small ranges use the serial

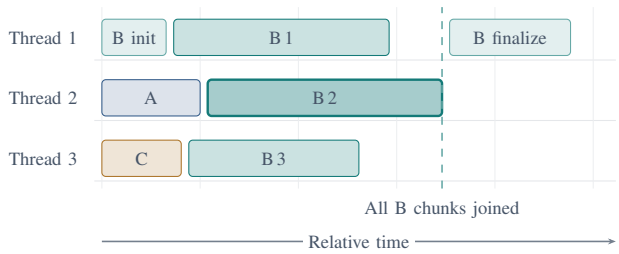


Fig. 8. One possible execution of Figure 7. Chunks from one sheet can run on different threads before ordered assembly. Bar lengths are schematic; individual steal events are not recorded.

path. Each chunk receives isolated metrics, and results are assembled in source order.

Ready chunks participate in admission decisions for new worksheets. A smaller sheet can yield admission while larger chunk work is pending. Batches retain only a bounded number of chunk results while the assembled output grows. Workbook contexts, the output buffer and deferred candidates remain additional memory consumers. Thread count, active-workbook count and writer count are separate experimental parameters.

Changed worksheet members are precompressed in parallel and handed to a workbook writer. The writer owns ZIP mutation and publishes a completed partial output only after processing succeeds. Unaffected members can reuse compressed bytes. If all entries are original, the source workbook is copied byte for byte. A fast unchanged-entry classification can use CRC32 and uncompressed length. That shortcut is collision-prone, and exact byte verification is available. Preservation and timing experiments must record which identity policy was used.

The method retains existing style and shared-string indices rather than globally renumbering their tables. This avoids unrelated rewrites and permits copy-through of unaffected members. It also leaves reductions requiring native save or global table compaction outside the cleanup subject.

IV. Evaluation

The evaluation asks three questions. RQ1 examines compressed package reduction and the remaining preservation footprint. RQ2 measures cleanup time, memory and the combined cleanup-plus-opening operation. RQ3 examines structural preservation and observed native presentation under the stated validation scope.

A. Corpus identity and outcome accounting

The evaluation treats Data.gov, Destatis and Enron as distinct populations. Archive checksums, member names and duplicate occurrences identify each selected corpus version. Legacy XLS conversion is reported separately from XLSX cleanup. Recursive extraction of the EDRM Enron v2 XML archives from Internet Archive inspected nested archives and embedded containers, yielding 34,065 distinct SHA-256-addressed candidates, including legacy Excel and other OLE objects. This input frame is separate from the prepared Enron XLSX population evaluated below.

The native visual stress cohort selects the 49 whole workbooks with the largest absolute compressed-file byte reduction in each corpus, totaling 147 original identities. All processor baselines use those same identities. Preparation, cleanup and visual-comparison outcomes are counted separately.

The frozen XLSX inventories contain 9,117 Data.gov members totaling 25.273 GB and 1,182 Destatis members totaling 2.364 GB (compressed file bytes, decimal GB); each archive member has a SHA-256 identity. Reported outcomes distinguish completed comparisons, differences, failures, unsupported representations and unfinished work, with retries counted separately. Exact package identity can bypass visual comparison; failed captures provide no equality observation.

B. Cleanup screening outcomes

Fresh release-build runs produced 1,182 Destatis, 8,623 Data.gov, and 30,301 prepared Enron XLSX outputs. Across those outputs, aggregate compressed file bytes fell by 15.48%, 5.27%, and 18.87%, respectively. The smaller, equal-size, and larger output counts were 1,045/30/107 for Destatis, 4,755/3,522/346 for Data.gov, and 21,565/8,415/321 for Enron. Hash checks confirmed byte identity for all equal-size outputs in these runs. Data.gov also had 494 unsupported-namespace skips. With sixteen workbook slots and sixteen Rayon threads, elapsed times were 119.7, 1,134.5, and 181.6 s, respectively, including analysis statistics and full replay events alongside concurrent native validation [13]. All three runs reached completion with zero reported processing failures. The host used an AMD Ryzen 7 7800X3D with eight physical cores, sixteen logical processors, 95.1 GiB OS-visible RAM and Windows 11 Home build 26200. Interactive replays accompany the Destatis, Data.gov and prepared Enron XLSX runs; replay clocks additionally include final reporting. Successful processing of these inputs does not establish correctness for every valid serialization. These descriptive results include files that grew and use only processed input bytes as the denominator. Enron's denominator begins after preparation. Visual preservation is evaluated separately. Figure 9 shows the workbook and worksheet distributions for these runs. Empty-cell cleanup and trailing layout rows account for prominent worksheet XML reductions. The contribution of column range merging is especially visible in Destatis. The worksheet panels exclude 88 prepared Enron workbooks without worksheet XML observations. Workbook payload sizes use all uncompressed ZIP member sizes. Among 9,117 Data.gov inputs, 493 workbooks contained explicitly prefixed worksheet data elements. After experimental namespace normalization and Default cleanup, only two produced smaller compressed packages, by 6.415% and 0.047% relative to the normalized baseline. Cell-record removal in other outputs did not reduce package size [14].

Corpus-driven development exposed both recognition and preservation defects. Earlier Enron controls isolated deleted center-across-selection spans and fill masks by restoring only

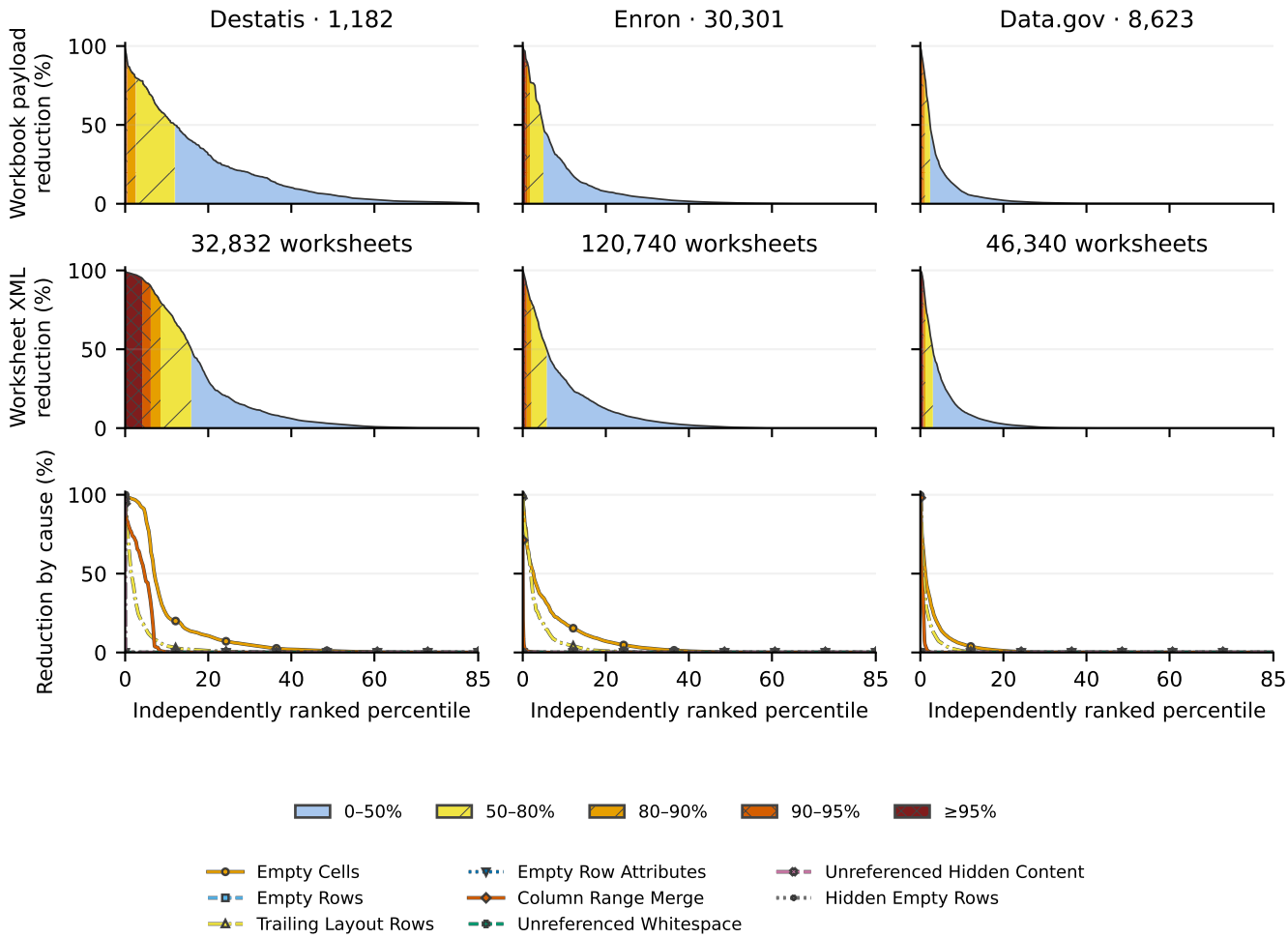


Fig. 9. Uncompressed-size reductions across the processed corpora. Rows show complete workbook payload, worksheet XML and per-cause worksheet XML reductions. Curves independently rank observations by decreasing reduction. The display ends at the 85th percentile; remaining positive reductions are below 0.5 percentage points. Colors and hatching identify reduction tiers. Data.gov namespace skips are excluded. Enron starts from prepared XLSX files; worksheet panels exclude 88 workbooks without worksheet XML.

the affected original cell records. These findings motivate retained fixtures and native controls alongside the screening counts [15].

C. Sampling and reference controls

The random evaluation sample is selected from a frozen eligible frame with a recorded seed before inspecting visual findings. Strata use corpus identity, package size and structural features available before validation. Low-reduction and unchanged files remain represented. A separately labelled development stress set contains large and historically problematic workbooks.

Feasible sampled files receive full native reference captures over the relevant visual area using a capture plan independent of the candidate selector. A full-reference timeout remains in the denominator. It is not replaced silently by an easier workbook. Mutations with known visible effects cover cell values, styles, geometry, conditional dependencies and objects. Mutations that Excel normalizes away are reported separately.

D. Performance and comparison subjects

We compared direct open–save trials with fresh six-worker SheetShrink cleanup followed by the same library. Across the two displayed laptop workbooks, all 33 SheetShrink trials completed under a 4 GiB commit limit, compared with 21 original trials. Median elapsed time improved in all seven fully completed pairs. Migration with openpyxl took 111.3 s directly and 9.7 s including cleanup.

Figures 10 and 11 show payload reduction and subsequent processing for selected cases. Full library matrices, absolute payloads and per-trial phases remain in the artifact. The laptop experiment retains 288 attempts; Enron adds eighteen attempts on the larger-memory host. SheetShrink payloads use separate controls whose member manifests match the native candidates; timed fresh outputs lack full per-trial member inventories.

Visual validation incomplete. Complete native visual comparisons remain pending for the case-study variants.

Figure 11 reports selected open–save endpoints through

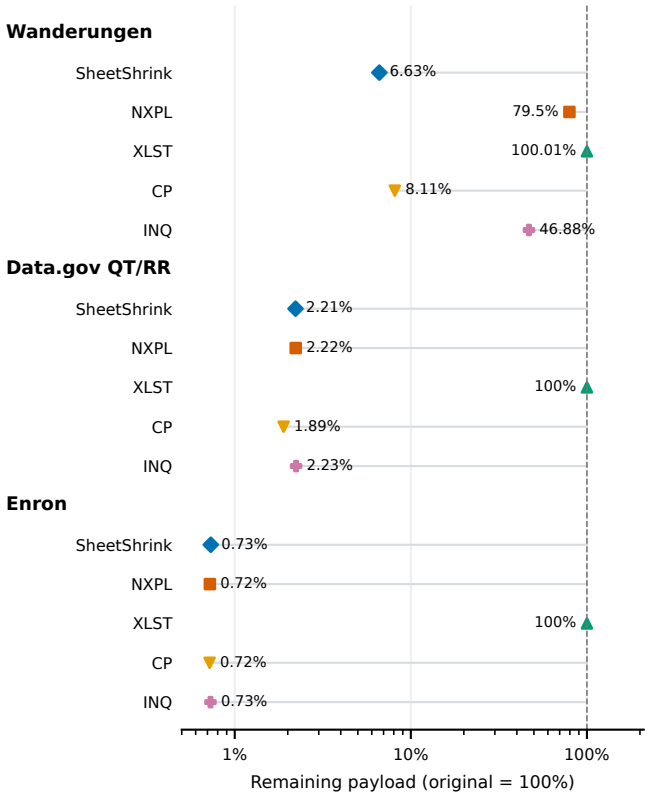


Fig. 10. Remaining uncompressed payload of the selected workbooks, relative to all ZIP-member bytes in the original. NXPL, XLST, CP and INQ denote NXPowerLite, XLStylesTool, Check Performance and Inquire. Lower is smaller; the x-axis is logarithmic. Original payloads are 248.39 MB (Wanderungen), 1,350.77 MB (QT/RR) and 353.15 MB (Enron). SheetShrink uses separately regenerated controls with verified native-candidate member identities. Tiny increases can round to 100%. Payload reduction and native visual preservation have separate evidence.

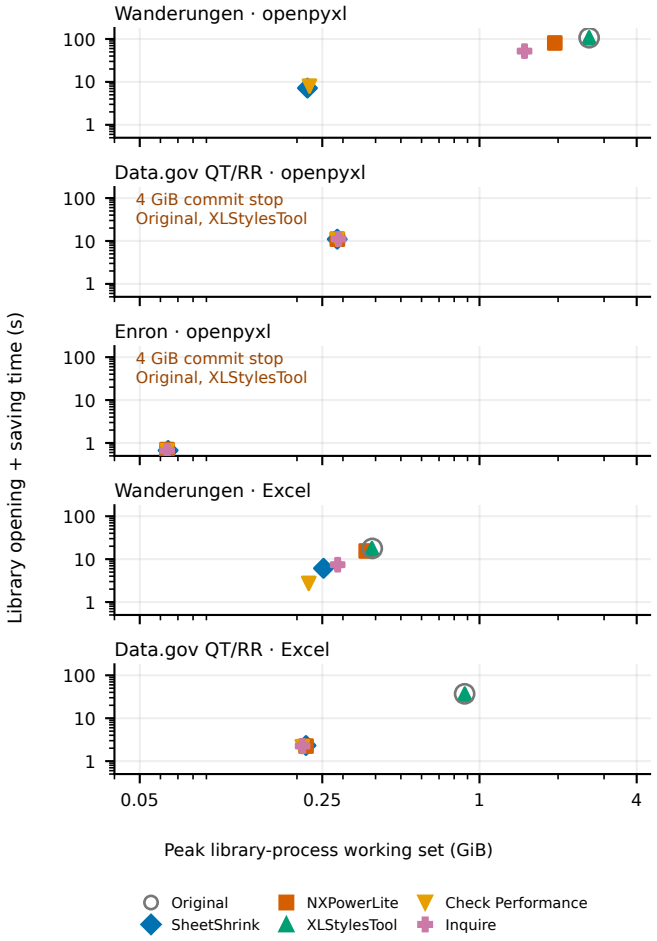


Fig. 11. Library open–save cost on selected workbooks, excluding cleanup. Points show median elapsed time and maximum library-process working set over three attempts; vertical bars show the elapsed-time range. Memory-stopped attempts have no completed-time point. SheetShrink uses fresh outputs; other tools use prepared outputs. Wanderungen and QT/RR use the laptop, Enron the larger-memory host. All jobs have a 4 GiB commit limit, distinct from working set. Comparisons are within panels.

library opening-plus-saving time and library-process working-set peaks. The artifact separately retains pipeline working sets and job-commit peaks. Each plotted time is a median of completed attempts, with memory maxima over all three attempts. Stopped attempts retain their elapsed time and memory boundary. Cleanup time is excluded uniformly from the figure. Fresh SheetShrink outputs and prepared external-tool outputs are processed by the same library endpoint. Pipeline coordinators are excluded from the plotted process-memory peaks. Fresh cleanup-plus-processing totals remain separately available in the artifact. The common host is a Ryzen 5 4500U laptop with six logical processors, 8 GiB installed RAM and Windows 11. Each job has a 4 GiB commit cap and a 900 s timeout, with the free-RAM reserve disabled. The SheetShrink executable is frozen at revision `765c480` and uses six Rayon and six writer workers. Native visual observations use the retained paired-rebuild protocol.

E. Larger-memory-host follow-up

The four displayed workbook/library combinations that reached the laptop's 4 GiB commit ceiling were repeated on an AMD Ryzen 7 7800X3D host with sixteen logical processors and 102,066,212,864 OS-reported physical bytes. Each combination has three Original and three fresh SheetShrink pipeline attempts at both 4 GiB and 24 GiB aggregate process-tree commit limits, totaling 48 displayed attempts. The same frozen SheetShrink binary uses sixteen Rayon and sixteen writer workers with one active workbook. The controller maintains an 8 GiB reserve for physical RAM and commit headroom, with a 900-second per-trial timeout. Python 3.12.14, openpyxl 3.1.5, pandas 3.0.1, R 4.6.2, readxl 1.6.0 and writexl 2.0.1 identify the tested software lane [16].

At 4 GiB, all twelve Original attempts reached the memory limit, while all twelve fresh-cleanup pipelines completed. At 24 GiB, the six Original attempts for Migration/readxl and QT/RR/openpyxl completed. Migration's two pandas endpoints reached the memory limit in all six attempts. All twelve fresh-cleanup pipelines completed at that limit. The artifact retains all completed and memory-limited endpoints. A

memory-stop termination time receives no completed-runtime ratio. The completed Migration/readxl and QT/RR/openpyxl pipelines were 2.97 and 24.28 times faster, respectively (392.149/131.953 s and 279.638/11.519 s for Original/fresh cleanup plus processing).

The larger limit distinguishes memory censoring from completed processing cost for these selected cases. Native visual comparisons, a separate cleanup benchmark and read-only evidence analysis ran concurrently. Filesystem caches were retained. The measured ratios are descriptive paired results under that context. Across all 48 displayed trials, sampled physical RAM and commit headroom remained above the declared system reserve.

After the timing series, separately regenerated controls for both displayed inputs matched every uncompressed member name, length and SHA-256 of the retained native candidate identities. Migration's native observation uses a documented normalized baseline, and QT/RR's retained selected plan has zero captures. The full per-trial member inventories were not retained for the timed fresh outputs, whose archive hashes remain in the receipts. A second cleanup was measured after each completed fresh pipeline, outside the timed endpoint. Its recorded ZIP identity flags do not establish general cleanup idempotence.

F. Enron selected-case follow-up

The Enron case uses the Enron workbook whose original SHA-256 begins `7298df9a6a76`. Its original archive contains 42,710,945 compressed bytes and 353,151,842 uncompressed member bytes. On the same larger-memory host, six variants each received three openpyxl 3.1.5 round-trip attempts, with a 4 GiB process-tree commit limit, an 8 GiB physical-RAM and commit reserve and a 300-second timeout [17]. Original and XLStylesTool reached the memory limit in all three attempts. Fresh SheetShrink cleanup followed by the library round trip completed with a median process-creation-to-exit time of 3.021 seconds and a maximum job-commit peak of 891.4 MiB. Prepared Check Performance, Inquire and NXPowerLite outputs completed their library round trips with respective medians of 1.036, 1.054 and 1.101 seconds, and maximum job-commit peaks of approximately 535.6 MiB.

The external prepared-output endpoints omit their tools' preparation cost. The SheetShrink endpoint includes fresh cleanup, so these observations do not establish an end-to-end speed ranking among cleaners. Memory-limited attempts remain censored. Recorded commit peaks slightly exceed the configured 4 GiB boundary and are retained without clamping. Concurrent native comparisons and retained filesystem caches qualify this selected-case result. After the timed series, a separate fresh SheetShrink control matched all eighteen uncompressed member names, lengths and hashes of the retained native candidate. Its output has 384,724 compressed bytes and 2,581,673 uncompressed member bytes. This representative identity check links the control to the visual audit, while preserving that audit's selected-coverage limitations. The timed trials retain archive hashes, without full per-trial member inventories.

G. Distributed comparator measurements

The native comparison campaigns are distributed across Windows hosts to obtain coverage within the available experimental budget. Native opening, cleanup and saving can dominate the cost of large spreadsheet collections. Each observation therefore retains its host, source identity, configuration and measurement boundary. Raw times remain associated with that host. Cross-tool ratios pair the same original workbook with a SheetShrink measurement on the same host. Host-specific ratios retain differences in workload, tool policy and session conditions. Each host and tool remains a separate plotted group. The frozen selection contains the 49 workbooks with the largest absolute compressed XLSX byte savings under the earlier SheetShrink screening in each of Destatis, Enron and Data.gov. The timing and preservation outcomes of this extreme-case panel apply to its selected inputs.

The present NXPowerLite lane uses a Lenovo 81YQ laptop with an AMD Ryzen 5 4500U (six physical cores and six logical processors), 8 GiB installed RAM (7.37 GiB OS-visible), a 512 GB Samsung MZALQ512HALU NVMe SSD and Windows 11 Home, build 26200. The recorded Windows power scheme is Energy Star. Other open Excel and validation processes are retained in the resource receipts.

a) Comparator configuration: NXPowerLite Desktop 11.0.1 x64 uses the custom profile `Cell formatting focused - Top49`. Excess cell-formatting cleanup is enabled. In the Excel profile, image resizing, JPEG conversion, cropped-area deletion, embedded-object flattening, PNG color-depth reduction and embedded chart-data removal are disabled. Lossy image modification conflicts with the objective of exact visible preservation and is deliberately excluded from the comparison policy. Verification of unchanged images requires the output-level audit.

Consequently, every retained output is audited against the original bytes of media, drawings, charts and embedded objects. Of 147 attempted workbooks, 129 produced reduced packages and ten explicitly remained unchanged. Eight Enron inputs reported an internal NXPowerLite error on both the primary attempt and a diagnostic repeat after restarting the application. Five outputs recompressed PNG images with identical decoded RGBA pixels and image dimensions. A sixth changed an embedded Word document. These six outputs are retained separately and excluded from the formatting-focused timing figure. The remaining 133 outputs passed the protected-part audit. Native worksheet equivalence requires separate visual validation.

XLStylesTool UWP 3.0.1.0 supplies an additional same-host comparison with forced style normalization, name unhiding and wholesale external-name removal disabled. The earlier conversion queue retains 122 hash-matched inputs with a confirmed primary conversion event and its complete per-attempt resource receipt. The other 25 Top49 inputs lack that complete timing evidence and remain outside the timing figure.

All 122 included outputs preserve the audited media and object parts byte-for-byte. The joint figure uses the intersection of 110 original workbooks, comprising 21 Destatis, 41 Enron and 48 Data.gov inputs. Both tools therefore use the same original files and repeated SheetShrink baselines within each plotted corpus group. Table I retains the full 133-input NXPowerLite resource lane.

Check Performance uses Excel 16.0.20527.20150 on the Lenovo host, with one owned private-desktop Excel worker per input. Among the same 110 originals, 102 primary workflows completed (86 confirmed Optimize All actions and sixteen no-optimization-needed outcomes), one was partial and seven failed, including one timeout. Complete outcomes comprise 21 Destatis, 36 Enron and 45 Data.gov inputs. The partial attempt saved an output before a progress-record replacement failed; its successful diagnostic retry remains separate from primary statistics. All original hashes matched before and after processing.

Check Performance wall time covers native-wrapper startup through exit, including control saves, optimization, artifact creation and shutdown. Input staging and independent hashing are excluded. Optimization and its associated saves share one phase; their individual times are unavailable. Excel-only CPU was retained through exit for all complete primaries, with working set and private bytes sampled at a median mean interval of 251 ms. All 990 reused Lenovo SheetShrink measurements list a concurrent Check Performance runner process; these snapshots do not establish whether it was active. The median descriptive workflow ratios are 20.19, 22.75 and 4.84 for Destatis, Enron and Data.gov. Background-load uncertainty and different endpoints qualify these ratios.

The Inquire lane uses an HP EliteBook 645 G11 with an AMD Ryzen 5 PRO 7535U (six physical cores, twelve logical processors), 32 GiB installed RAM, a 512 GB SOLIDIGM NVMe SSD and Windows 11 Enterprise build 26200.9711. Excel and Inquire file versions are 16.0.17932.21056 and 16.0.16626.20076. Of the same 110 originals, 90 have full primary Inquire successes, six partial outcomes and fourteen failures. Diagnostic retries remain separate. Full successes comprise 19 Destatis, 28 Enron and 43 Data.gov workbooks.

Inquire records single historical Excel workflow observations, including opening, cleanup with internal saving, explicit saving and shutdown; detailed boundaries remain in the per-case receipts. Partial and failed attempts receive no completed-runtime ratio. Historical hardware and version attribution partly uses execution-session provenance and later inventory rather than per-trial capture. Among the 90 complete pairs, 41 Inquire work directories were on local NVMe, 48 on a USB-connected Samsung 870 QVO SSD, and one has incomplete storage attribution. All paired SheetShrink trials used local NVMe. The source receipts retain these storage conditions. The median descriptive workflow ratios are 19.36, 15.18 and 12.04 for Destatis, Enron and Data.gov, respectively. Different endpoints and storage conditions constrain their interpretation.

The HP SheetShrink lane freezes revision `75e85035`, twelve Rayon and twelve writer workers, and one active workbook. One warm-up precedes seven measured trials per input, extended to eleven when the relative interquartile range exceeds 10%. All 110 inputs completed: 110 warm-ups and 806 measured trials, with identical uncompressed output contents across repetitions for each input. Trials used fresh native Windows processes, warm filesystem caches, seeded shuffled ordering and a 1,800 s timeout. Hashing and verification are outside process-start-to-exit timing. The host retained its institutional power scheme and background workload. No comparator Excel process ran concurrently with these timed SheetShrink trials.

b) Timing and resources: Each NXPowerLite and XL-StylesTool observation has one primary timing measurement per input. Its operational cost limits replication, and its timing variability therefore remains unestimated. Diagnostic retries remain separate attempts. Elapsed time covers automated GUI orchestration through output confirmation and initial package verification. CPU measurements accumulate attributed process-tree user and kernel time. Working-set and private-byte peaks use a nominal 250 ms sampling interval. The existing warm application is reused, with one conversion worker and a 512 MiB RAM and commit reserve.

The Lenovo SheetShrink experiment uses an optimized release build from revision `765c480`, one active workbook, one Rayon worker and one writer, with the default cleanup policy. One complete warm-up round precedes nine measured rounds over the same 147 source identities. Each round independently shuffles workbook order with a recorded seed. Each file starts a fresh CLI process, and measured wall time ends at process exit. Output hashing is outside this timing boundary. Filesystem caches are left unflushed after warm-up. Direct-child CPU and OS peak-working-set counters are retained, with private usage sampled at a nominal 10 ms polling interval. These memory observations have different sampling semantics from the GUI lane and are reported separately. All 1,470 invocations completed successfully. Each input produced the same package hash across warm-up and measured repetitions, and all 147 final outputs preserved the original bytes of the audited media and object parts. Across inputs, the median relative within-input interquartile range, $(Q_3 - Q_1)/\widetilde{t}$, was 0.84% over the nine measured repetitions.

Figure 12 retains raw times and workbook-level same-host ratios. Each denominator is the same-host SheetShrink median (nine Lenovo trials or seven to eleven HP trials), and each numerator is the primary comparator observation. Across-workbook quartiles describe heterogeneity in this deliberately selected stress panel. They are distinct from uncertainty over repeated trials [18]. The ratio includes the stated interface and confirmation costs, and the tools have different cleanup policies. It therefore measures the observed cleanup workflows. Opening-time improvements and cleanup-plus-opening costs require their own readiness endpoints, as in the separate historical opening experiment.

TABLE I
MEDIAN RESOURCE OBSERVATIONS ACROSS PAIRED WORKBOOKS ON THE LAPTOP. CPU IS PROCESS TIME IN SECONDS. WS IS WORKING SET IN MIB, USING OS PEAK COUNTERS FOR SHEETSHRINK AND SAMPLED PEAKS FOR NXPOWERLITE.

Corpus	Pairs	CPU SheetShrink	CPU NXPowerLite	WS SheetShrink	WS NXPowerLite
Destatis	44	1.24	5.45	54	248
Enron	41	0.70	4.19	51	306
Data.gov	48	6.73	10.49	415	146

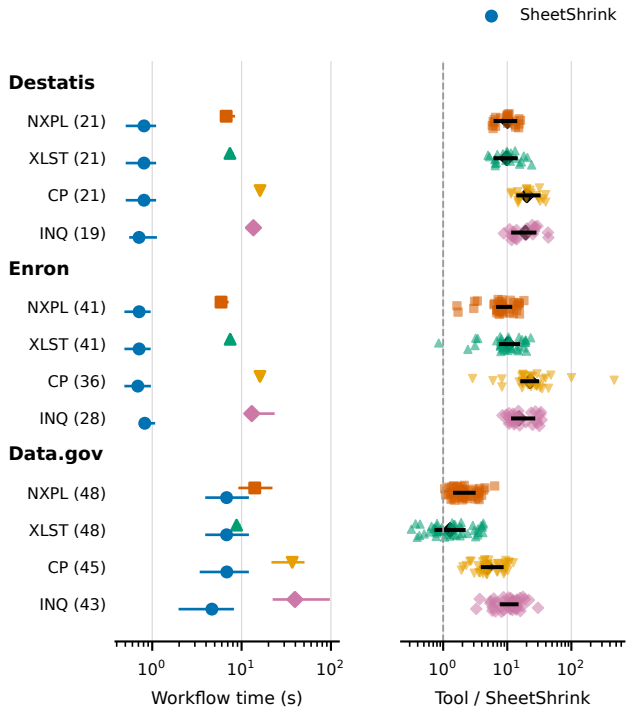


Fig. 12. Cleanup times and tool/SheetShrink ratios by corpus; parentheses give pair counts. Time summaries and black ratio marks show medians and interquartile ranges. NXPL: NXPowerLite; XLST: XLStylesTool; CP: Check Performance; INQ: Inquire.

H. Native validation of preservation rules

a) Row-height preservation.: In the Traffic workbook, promoting repeated empty-cell formatting into a row style changed an automatically determined height from 12.75 to 9 points, shifting subsequent content and changing 3,408 pixels in an independently specified reference region. Requiring an explicit custom height before promotion restored byte-identical native captures [10].

b) Font representatives.: Two Destatis cases validate complementary font-preservation conditions. In a higher-education workbook, removing 87 blank cells carrying explicit default-font and no-fill metadata changed 25,640 pixels despite equal capture geometry and a pixel-identical original-versus-original control. Retaining the required representatives restored equality in an independently specified reference region. Legacy print-name aliases were normalized for opening and recorded separately. In a cultural-finance workbook, a blank cell's non-default font affected automatic row height despite an omitted `applyFont` flag; a stored height without `customHeight` did not prevent the change from 15 to 12.75 points. Retaining the representative restored pixel and geometry equality in the independent control and all 350 regions selected by the targeted capture policy.

c) Background definitions.: In a Destatis horticulture workbook, SheetShrink reduced column definitions from 128,912 to 2,197. A corrected DeltaSieve background-envelope guard reduced the capture selection from 16,101 to 165 regions; all 165 resulting image pairs were pixel-identical.

d) Validation boundaries.: In Wanderungen, excluding an empty trailing context row removed a spurious height finding. All 37 captured image pairs were pixel-identical under the DeltaSieve capture policy, while separate effective-height observations on two other worksheets remained unresolved [19]. These case studies validate specific preservation conditions within the recorded capture scopes; they do not establish exhaustive workbook or corpus-wide visual equivalence.

I. Targeted visual outcomes on the Top49 selection

A version-matched audit of the 49 workbooks with the largest absolute SheetShrink reductions in each corpus retained 42 Destatis, 46 Data.gov, and 46 Enron comparisons against the selected originals. Separately documented normalized baselines supplied five further Destatis and two further Enron observations. The remaining two, three, and one slots, respectively, had no eligible observation. Across both baseline categories, zero Destatis, seven Data.gov, and five Enron workbooks had recorded differences. These counts describe the selected capture scope and include findings whose cause remains unresolved. The five Enron workbooks had six chart findings of 2–26 changed pixels with equal image dimensions and no reported geometry change. The missing Enron workbook required repair when opened in Excel, including after a separate legacy reconversion.

J. Evidence boundary

The selected controls demonstrate concrete preservation conditions and recorded failures. They do not estimate corpus-wide defect prevalence. Reduction results cover the frozen XLSX inventories. Matched cleanup-plus-opening measurements and independent native references cover selected cases. Broader reference coverage and matched whole-corpus cost remain open. Detailed development attempts, input hashes, native images and producer versions remain in the source-linked artifact.

V. Related Work

A. Workbook bloat and cleanup tools

Microsoft Check Performance removes cells associated with unnecessary formatting and other stored information from a loaded workbook [20]. Its documentation warns that optimization can remove pixel-art cells. Inquire's Clean Excess Cell Formatting command removes excess cells beyond the last nonblank cell and can change conditional-formatting coverage [21]. O'Beirne's published analysis of Excel 2013 Inquire discusses this command and criticizes its lack of information about planned and completed removals [22].

Package-level cleanup also predates SheetShrink. XLStylesTool's developer describes an Open-XML-based cleaner for excessive and corrupted styles that operates independently of Excel's object model [23]. NXPowerLite Desktop 10 lists excess XLSX cell-format cleanup among its features [24]. Its engine description discusses formatted blank cells as a source of large worksheet parts [25]. Their available descriptions do not establish the guarded removal rules, compact row-font witnesses or native preservation outcomes examined here. SheetShrink's positioning therefore includes its effect-aware rules, retained representatives and measured costs alongside its execution before native loading.

B. Spreadsheet scalability and system optimization

Mack et al.'s CHI Extended Abstracts case study characterizes scalability problems reported in spreadsheet forums [26]. Rahman et al. benchmark Excel, LibreOffice Calc and Google Sheets on spreadsheet computations and examine indexing, data layout, incremental evaluation and shared computation [27]. Their workloads assess system responsiveness and computation, while SheetShrink measures removable persisted representation and the cost of preparing workbooks for subsequent use.

DataSpread's storage engine integrates spreadsheet presentation with database storage and positional access [28]. Anti-Freeze schedules asynchronous formula computation to retain interactivity during updates [29]. Hillview distributes exploration of large datasets and uses compact visualization sketches [30]. TACO exploits regular formula-reference patterns to compact dependency graphs and query them directly [31]. SheetShrink transforms an existing XLSX package into another editable workbook under a stated preservation contract.

C. Spreadsheet loading and specialized parsing

SheetReader studies specialized XML parsing for efficient spreadsheet ingestion, including the coupling of decompression, parsing and parallel execution [1]. During baseline ingestion, we encountered an empty-cell parsing defect in SheetReader and submitted a correction with regression fixtures [32].

SheetReader loads worksheet data into analytical representations. SheetShrink rewrites an editable workbook whose required contents, dependencies and presentation must survive cleanup. Its byte-oriented scanner therefore combines XML recognition with preservation-specific guards. Corpus-scale processing and targeted regression controls exercise this implementation within its supported domain, while additional serializations and semantic cases can remain untested.

D. Spreadsheet refactoring and validation

Hermans et al. detect formula smells and suggest refactorings to improve spreadsheet maintainability [33]. SheetShrink's removal rules additionally account for formatting inheritance and native layout effects of stored cells.

Spreadsheet slicing integrates dependency reasoning with testing and debugging [34]. Dependencies also constrain removal when retained formulas or graphical objects refer to candidate content. Workbook comparison tools provide observations about cell and format differences [35]. Our native validation protocol uses DeltaSieve's region selection and capture workflow [4], with independent references and mutation controls to evaluate selected coverage.

VI. Discussion and Limitations

The preservation contract is narrower than all possible spreadsheet behavior. Rendering depends on the installed Excel version, fonts and calculation state. A selected-region observation has limited scope. Unsupported XML and unsuccessful opening or saving remain visible outcomes. The independent reference sample estimates behavior on its eligible population and cannot establish a universal absence of regressions.

Large reductions can coexist with small preservation witnesses. Font metadata, inheritance masks and content-bearing hidden tails therefore require separate conditions. The constant-tail policy remains a structural heuristic. The fast CRC/length identity shortcut is collision-prone, and the selected verification mode must accompany measurements. The built-in-name predicate also lacks a formula-body equivalence check.

VII. Conclusion

SheetShrink addresses spreadsheet redundancy through effect-aware removal and compact preservation. Its implementation and evaluation distinguish compressed size, XML changes, execution cost and observed presentation.

Acknowledgment of AI Assistance

OpenAI ChatGPT and Codex assisted with drafting and editing the Abstract and Sections I–VII, literature discovery and organization, implementation and refactoring, test and experiment scripts, figure-generation code, and analysis of retained experiment evidence. This assistance included the method diagrams, XML examples and execution visualizations. Local agent workflows using these systems also supported overnight validation campaigns by inspecting status and logs, investigating stalled attempts, and documenting candidate visual regressions for author review.

The author directed development, reviewed source code, and performed manual testing and debugging. Agent observations supported investigation. Reported validation outcomes

are grounded in the retained native captures and comparison records. The author remains responsible for the scientific claims, references, evidence selection, originality and final submitted content.

REFERENCES

[1] H. Gavriilidis, F. Henze, E. T. Zacharatou, and V. Markl, "SheetReader: Efficient specialized spreadsheet parsing," *Information Systems*, 2023.

[2] Microsoft, "Row.CustomHeight property," Open XML SDK documentation, accessed November 9, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.row.customheight

[3] SheetShrink project, "XLSX shrinking behavior," Versioned software specification, revision a7eef1a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/a7eef1a/docs/behavior/shrinker.md

[4] "DeltaSieve: Targeted visual validation of spreadsheet transformations," Companion working manuscript prepared for ICDE 2027, 2026, supplementary material, related-manuscripts/deltasieve.pdf.

[5] DeltaSieve project, "Native wrapping and merged-cell reference controls," Versioned experiment receipts, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/ab016bd2/docs/visual-validation/experiments/independent-reference/layout-controls-20261111.json

[6] ——, "INDIRECT conditional-format fallback controls," Versioned experiment receipts, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/6faefcaa/docs/visual-validation/experiments/independent-reference/dependency-controls-20261111.json

[7] SheetShrink project, "Preserve attributed cell value elements," Revision 88c77f9, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/commit/88c77f9557cd63d6ed887c5389ebb7e4a55452fc

[8] ——, "Preserve defined-name worksheet scope," Revision 5822800, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/commit/582280043a56a0346cb891fb0b9bbf23b4a3a875

[9] ——, "Do not publish outputs from incomplete worksheet parses," Revision c608d4e, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/commit/c608d4e5155fcbf584f9e2154a378c6f5fa42e2b

[10] ——, "Dense row promotion and inherited height," Versioned protocol and evidence, revision e5e4fe8, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/e5e4fe8/docs/visual-validation/experiments/traffic-row-height/README.md

[11] ——, "Hidden constant-tail investigation and wanderungen controls," Versioned evidence, revision 1b9ee98, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/1b9ee98/docs/visual-validation/investigations/hidden-constant-tails-20261104.md

[12] ——, "Xlsx shrinking behavior," Versioned protocol and evidence, revision 7ca7c2a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/7ca7c2a/docs/behavior/shrinker.md

[13] ——, "Complete-corpus distribution and replay measurements," Versioned evidence, revision ce3722cb, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/ce3722cb/docs/visual-validation/experiments/full-corpus-distribution-20261111.json

[14] ——, "Prefixed worksheet data inventory," Inventory, normalization trials and sampled native controls, revision 44d73f3, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/44d73f3/docs/visual-validation/experiments/prefixed-data/README.md

[15] ——, "Confirmed default visual regressions," Controlled restoration evidence, revision 1b9ee98, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/1b9ee98/docs/visual-validation/confirmed-default-regressions.md

[16] "Bounded opening and saving follow-up," Supplementary experimental artifact, 2026, protocol, individual receipts, source snapshot and candidate-identity controls in evidence/bounded-open-save-followup.

[17] "Enron selected-case opening and saving follow-up," Supplementary experimental artifact, 2026, eighteen bounded receipts, protocol and representative native candidate identity control in evidence/enron-open-save-followup.

[18] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 2015, pp. 73:1–73:12.

[19] DeltaSieve project, "Targeted visual validation behavior, empty trailing row exclusion," Versioned protocol and evidence, revision 7ca7c2a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/7ca7c2a/docs/behavior/targeted-validation.md

[20] Microsoft, "Cleanup cells in your workbook," accessed November 9, 2026. [Online]. Available: https://support.microsoft.com/en-us/excel/cleanup-cells-in-your-workbook

[21] ——, "Clean excess cell formatting on a worksheet in Excel for Windows," accessed November 9, 2026. [Online]. Available: https://support.microsoft.com/en-us/excel/clean-excess-cell-formatting-on-a-worksheet-in-excel-for-windows

[22] P. O'Beirne, "Excel 2013 Spreadsheet Inquire," in *Proceedings of the EuSpRIG 2013 Conference "Spreadsheet Risk Management"*. European Spreadsheet Risks Interest Group, 2013, also available as arXiv:1401.7586, deposited in 2014. [Online]. Available: https://eusprig.org/wp-content/uploads/1401.7586.pdf

[23] XLGeek, ".NET4.0 version of the XLStylesTool is now available," 2011, developer description, accessed November 9, 2026. [Online]. Available: https://sergeig888.wordpress.com/2011/03/21/net4-0-version-of-the-xlstylestool-is-now-available/

[24] M. Power, "What's new in NXPowerLite Desktop 10," vendor release description, accessed November 9, 2026. [Online]. Available: https://neuxpower.com/blog/whats-new-in-nxpowerlite-desktop-10

[25] ——, "Compression engine major upgrade—version 10," vendor description, accessed November 9, 2026. [Online]. Available: https://neuxpower.com/blog/wecompress-file-compressor-engine-upgrade

[26] K. Mack, J. Lee, K. Chang, K. Karahalios, and A. Parameswaran, "Characterizing scalability issues in spreadsheet software using online forums," in *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, 2018. [Online]. Available: https://arxiv.org/abs/1801.03829

[27] S. Rahman, K. Mack, M. Bendre, R. Zhang, K. Karahalios, and A. Parameswaran, "Benchmarking spreadsheet systems," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, 2020, pp. 1589–1599. [Online]. Available: https://sajjadur.net/files/benchmarking_spreadsheets.pdf

[28] M. Bendre, V. Venkataraman, X. Zhou, K. C.-C. Chang, and A. Parameswaran, "Towards a holistic integration of spreadsheets with databases: A scalable storage engine for presentational data management," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 113–124. [Online]. Available: https://arxiv.org/abs/1708.06712

[29] M. Bendre, T. Wattanawaroon, K. Mack, K. Chang, and A. Parameswaran, "Anti-Freeze for large and complex spreadsheets: Asynchronous formula computation," in *Proceedings of the 2019 International Conference on Management of Data*. ACM, 2019, pp. 1277–1294. [Online]. Available: https://people.eecs.berkeley.edu/~adityagp/papers/dataspread-async.pdf

[30] M. Budiu, P. Gopalan, L. Suresh, U. Wieder, H. Kruiger, and M. K. Aguilera, "Hillview: A trillion-cell spreadsheet for big data," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1442–1457, 2019. [Online]. Available: https://arxiv.org/abs/1907.04827

[31] D. Tang, F. Chen, C. De Leon, T. Wattanawaroon, J. Yun, S. Seshadri, and A. G. Parameswaran, "Efficient and compact spreadsheet formula graphs," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 2634–2646. [Online]. Available: https://arxiv.org/abs/2302.05482

[32] A. Aue-Johr, "Fix empty-cell parsing and preserve following cell positions," SheetReader core, pull request 5, submitted November 11, 2026. [Online]. Available: https://github.com/polydbms/sheetreader-core/pull/5

[33] F. Hermans, M. Pinzger, and A. van Deursen, "Detecting and refactoring code smells in spreadsheet formulas," *Empirical Software Engineering*, vol. 20, no. 2, pp. 549–575, 2015.

[34] J. Reichwein, G. Rothermel, and M. Burnett, "Slicing spreadsheets: An integrated methodology for spreadsheet testing

and debugging," in *Proceedings of the Second Conference on Domain-Specific Languages*. USENIX, 1999, pp. 25–38. [Online]. Available: https://www.usenix.org/legacy/publications/library/proceedings/dsl99/full_papers/reichwein/reichwein_html/index.html

[35] Microsoft, "Basic tasks in spreadsheet compare," Microsoft Support, accessed November 8, 2026. [Online]. Available: https://support.microsoft.com/en-gb/excel/basic-tasks-in-spreadsheet-compare